

(1.) План публикации продукта (MVP Birthsync)

(1.1.) Исходные допущения

Мы исходим из того, что публикуем MVP BirthSync: телеграм-бота.

- напоминания о днях рождения;
- анкеты друзей с предпочтениями;
- виш-листы и базовый обмен ими;
- учёт идей подарков.

Это важное допущение, потому что план публикации MVP принципиально отличается от плана полноценного «фиченаполненного» релиза. В MVP мы сознательно оставляем часть идей (например, сложные рейтинги подарков, AI-подбор и глубокую интеграцию с маркетплейсами) на последующие релизы, чтобы:

1. Сократить технический периметр возможных отказов в первые недели.
2. Быстрее выйти на рынок и начать собирать реальную обратную связь, а не гадать на опросах.
3. Не перегружать коммуникации: чем проще обещание, тем легче его выполнить без кризиса.

Мы также считаем, что к моменту старта у нас уже оформлена юридическая «коробка» ООО и базовая структура ролей и ответственности внутри команды (Мирослав — тимлид и гендиректор, Роман — техлид, Полина — дизайн и маркетинг, Виолетта — QA и аналитика).

(1.2.) Фазовая модель релиза

Вместо линейного «написали — выложили» мы строим итеративную, кризисоустойчивую модель из четырёх фаз:

1. Подготовка (−6...−2 недели до публичного релиза).
2. Технический «фриз» и мягкий запуск (soft-launch) (−2...0 недель).
3. Публичный релиз (неделя 0).
4. Стабилизация и цикл быстрой обратной связи (0...+4 недели).

Мы осознанно вводим soft-launch и пострелизный цикл как отдельные фазы, потому что реальный риск для стартапа — не столько «сломанный билд», сколько отсутствие управляемой реакции на баги и негатив.

(1.2.1.) Фаза подготовки (–6...–2 недели)

На этой фазе мы строим инфраструктуру релиза и подготавливаем коммуникации, не открывая продукт широким пользователям.

Технические шаги:

- Выстраиваем CI/CD на основе нашего репозитория GitHub: настроенные ветки main / develop, автоматическая сборка и прогон unit-тестов, сборка релизных билдов по тегам. Это снижает риск человеческой ошибки при сборке и ускоряет rollback, если кризис.
- Разворачиваем три окружения: dev, stage, prod. На dev живёт быстрый эксперимент, stage — предрелизная среда, максимально похожая на прод, prod пока закрыт (ограниченный доступ). Это нужно для того, чтобы любые баги ловить в stage, а не на живых пользователях.
- Подключаем аналитический контур: события активации, онбординга, создания анкет и виш-листов, добавления идей подарков, отправки напоминаний. Аналитику мы заводим ещё ДО маркетинга, потому что иначе любые деньги в рекламу — выстрел в темноту и потенциальный кризис бюджета.
- Настраиваем мониторинг и алёртинг (uptime, ошибки 5xx, время ответа API, падения мобильных приложений). Это фундамент для кризисного управления: Мирослав не может управлять ситуацией, если команда видит проблему через отзывы в сторе, а не через алёрты.

Коммуникационные шаги:

- Полина вместе с нами формирует коммуникационную рамку: как мы описываем проблему (стресс из-за дней рождения, выбор подарков), какую пользу обещаем и чего не обещаем в первой версии (чтобы не перегреть ожидания).
- Создаём лист ожидания (простая форма/лендинг), куда собираем первые e-mail/телеграм-контакты. Это важно, потому что к моменту релиза у нас уже есть «теплая» аудитория, и мы не зависим только от платного трафика.
- Готовим скелет лендинга, карточек приложения и базовый FAQ — это пригодится и для релиза, и для обработки первых негативных отзывов.

Распределение ответственности на фазе:

Направление	Кто владелец	Почему именно так
CI/CD, окружения, мониторинг	Роман	Технический контур — критичный риск; техлид отвечает за архитектуру и надёжность.
Событийная аналитика	Роман + Виолетта	Роман — корректность внедрения, Виолетта — смысловая целостность событий и дальнейший анализ.
Коммуникационная рамка	Полина	Это ядро бренда; от него зависят CTR, ожидания и тональность всех последующих сообщений.
Общий приоритизационный контроль	Мирослав	В случае конфликта приоритетов (фича vs стабильность) нужен один доминирующий голос.

Мы задаём длительность фазы в 4 недели (диапазон –6...–2), потому что для небольшой команды из четырёх человек это реалистичный срок на выстраивание инфраструктуры без параллельного сильного давления маркетинга. Если делать быстрее, сильно возрастает вероятность технической задолженности, которая выстрелит уже в кризисной ситуации, то есть в дни релиза.

(1.2.2). Фаза soft-launch (–2...0 недель)

Здесь мы сознательно открываем продукт ограниченному числу пользователей (например, 200–500 человек) и замедляем включение маркетинга.

Технические шаги:

- Замораживаем фичи (feature freeze): принимаем решение, что в релиз пойдёт только исправление багов и критичных UX-огрехов. Любая новая фича — после релиза. Это жёсткое ограничение, но оно снижает риск, что свежий неотестированный функционал «положит» прод в день объявления.
- Разворачиваем build soft-release в Telegram (закрытое тестирование/по инвайтам). Это позволяет «пощупать» реальную аудиторию, систему обновлений и поведение бота/мини-эппов на разных девайсах.
- Совместно с Виолеттой прогоняем план регрессионного тестирования: авторизация в боте, создание записи о человеке, напоминания, виш-лист, удаление аккаунта. Приоритет багов выстраиваем не по удобству команды, а по риску для репутации (например, баг «не приходит уведомление о ДР» критичнее, чем «на одном экране обрезана иконка»).

Коммуникации:

- Полина готовит серии писем/сообщений тестовой аудитории: приглашение, онбординг, запрос обратной связи через простую форму и отдельный канал (Telegram-чат или e-mail). Мы хотим, чтобы первые 200–500 человек ощущали себя не массовыми пользователями, а партнёрами в улучшении продукта.
- Мы заранее формируем шаблон кризисного сообщения для soft-launch: если обнаружился критический баг (например, потеря данных), Мирослав в течение 1–2 часов после выявления публикует прозрачное сообщение: что сломалось, что мы уже сделали, какие меры принимаем. Наличие готового шаблона экономит драгоценное время в реальном кризисе.

Почему именно 200–500 пользователей на soft-launch: это компромисс между статистической значимостью (нам нужны хотя бы сотни событий, чтобы увидеть поведение воронки) и управляемостью (если сразу придёт 5–10 тысяч человек, мы утонем в поддержке и не успеем реагировать).

(1.2.3.) Публичный релиз (неделя 0)

Под публичным релизом мы понимаем момент, когда:

- бот запущен, мини-эпп запущено;
- лендинг открыт;
- мы даём первые публичные анонсы в своих каналах и запускаем ограниченный платный трафик.

Технические действия:

- Включаем прод-мониторинг с агрессивными порогами: алёрты при росте ошибок 5xx/4xx, падении конверсии на ключевых шагах онбординга, росте количества падений приложений. Оповещения идут минимум в общий канал команды + личные уведомления Роману и Мирославу.
- Задаём правила оперативного rollback: если в течение, условно, часа после релиза мы фиксируем, что критический баг повторяется у значимой части пользователей (например, >10% попыток регистрации падают), Роман имеет право инициировать откат на предыдущую стабильную версию без отдельного голосования, а Мирослав параллельно инициирует коммуникацию о проблеме.

Коммуникации:

- Полина публикует основной анонс (о нём ниже в задании 2) в ключевых каналах: Telegram/соцсети, сайт, возможно, профильные сообщества.

- Мы ограничиваем стартовый маркетинговый бюджет в первые 3–5 дней (условно, до 20–30% от того, что планируем тратить в штатном режиме). Это сознательное решение: мы не хотим завести в систему большой поток пользователей, пока не увидим реальную стабильность и retention первых когорт.
- Виолетта с первого дня собирает ежедневные отчёты по метрикам (DAU, регистрация, завершение онбординга, создание хотя бы одной анкеты и одного виш-листа, отказоустойчивость).

Таким образом, публичный релиз у нас — не «фейерверк любой ценой», а управляемое расширение нагрузки.

(1.2.4.) Стабилизация и цикл обратной связи (0...+4 недели)

В течение первого месяца после релиза наша основная задача — стабилизировать продукт и превратить сырую обратную связь в структурированные решения.

Технические шаги:

- План небольших быстрых патч-релизов (раз в 7–10 дней) с критичными исправлениями и минимальными точечными улучшениями UX. Мы сознательно не делаем в этот период больших фич, чтобы не создавать дополнительную нестабильность.
- Вводим канал обратной связи в самом продукте (простая форма «Что улучшить?»), чтобы не потерять отзывы, которые пользователю лень писать на e-mail или в стор.

Коммуникации:

- Публикуем регулярные небольшие апдейты (например, раз в неделю) в формате: «что мы исправили, какие улучшения внесли, какие запросы пользователей взяли в работу». Это снижает уровень тревоги и показывает, что продукт живой, а не «выстрелил и замер».
- В случае серьёзного инцидента (потеря части данных, длительный даунтайм) работаем по заранее принятому кризисному сценарию: Мирослав — внешняя коммуникация, Роман — техническое восстановление, Полина — упаковка сообщений для соцсетей, Виолетта — пост-мортем: какие данные потеряны, что делаем, чтобы это не повторилось.

(1.3.) Таблица фаз, действий и ответственности

Чтобы назначение ответственности было прозрачным, фиксируем это в компактной таблице:

Фаза	Ключевые действия	Ответственные (ведёт → участвует)	Аргументация
Подготовка	CI/CD, окружения, аналитика, лендинг, лист ожидания	Тех: Роман → Виолетта; Комм: Полина → Мирослав	Техконтур должен быть выстроен под контролем техлида; коммуникации — под маркетингом.
Soft-launch	Feature freeze, закрытое тестирование, сбор ранней обратной связи	Тех: Роман → Виолетта; Комм: Полина → Мирослав	Фокус на стабильности; QA и аналитика важнее, чем добавление новых функций.
Публичный релиз	Активация мониторинга, ограниченный маркетинг, анонсы	Решения в кризис: Мирослав; Техреакция: Роман; Сообщения: Полина; Метрики: Виолетта	В кризис мы избегаем «советов четырёх человек»; финальное слово за тимлидом, техлид — за rollback'ами.
Стабилизация	Патчи, регулярные апдейты, пост-мортемы, формирование бэклога	Тех: Роман; Аналитика: Виолетта; Комм: Полина; Приоритеты: Мирослав	Мы превращаем хаотичный поток фидбэка в управляемый план релизов, опираясь на аналитические данные.

(2.) Подготовка «упаковки»: лендинг, карточка приложения и анонс

(2.1.) Логика «упаковки» Birthsync

У «упаковки» две функции:

1. Продать решение проблемы, а не набор экранов: человек испытывает стресс перед днями рождения, боится забыть, выбрать не тот подарок, потратить время на поиск.
2. Сразу снизить ожидания до реалистичных для MVP, чтобы не столкнуться с валом разочарований («обещали всё, а там только напоминания»).

Поэтому мы строим лендинг и карточки приложения вокруг одной простой идеи: «BirthSync — единое место, где мы заранее готовим подарки и поздравления так, чтобы в день X не было паники».

(2.2.) Структура лендинга

Мы сознательно делаем лендинг не перегруженным, но с понятной логикой блоков.

За основу данного блока взят отчёт:

<https://github.com/Birthsync/.github/blob/main/%D0%A1%D0%B5%D0%BC%D0%B5%D1%81%D1%82%D1%80%202/%D0%9F%D1%80%D0%B0%D0%BA%D1%82%D0%B8%D1%87%D0%B5%D1%81%D0%BA%D0%BE%D0%B5%20%D0%B7%D0%B0%D0%B4%D0%B0%D0%BD%D0%B8%D0%B5%202.md>

1. Hero-блок (главный экран).

Содержит название, краткий слоган, скриншот приложения и СТА.

Пример текста:

BirthSync. Наконец-то дни рождения перестают быть стрессом.

Храним важные даты, знаем предпочтения друзей, подбираем подарки заранее — а не ночью перед праздником.

СТА: кнопки «Скачать в App Store / Google Play» и «Оставить контакт» (если приложений в сторсах ещё нет).

Почему так: в первых двух строках мы артикулируем эмоциональную боль («стресс») и сразу показываем, что решаем её системно, а не «просто напоминаем о датах».

2. Блок «Проблема» (коротко).

Краткий текстовый блок, в котором мы описываем знакомую ситуацию: забытые дни рождения, панические поиски, неподходящие подарки.

Это нужно, чтобы пользователь узнал себя и понял: продукт создан не абстрактно, а под его реальные боли.

3. Блок «Как решает BirthSync».

Здесь мы описываем три–четыре ключевых преимущества, которые уже реализованы в MVP:

- Календарь праздников с напоминаниями.
- Анкеты друзей с важными деталями (размер, любимые бренды, три «нельзя» и т.п.).
- Виш-листы с возможностью сохранять ссылки на магазины.
- Простое планирование подарков «на будущее» (идея с датой и ориентировочным бюджетом).

Мы сознательно не перегружаем этот блок списками второстепенных фич. Важнее показать, что всё, что нужно, действительно находится в одном месте, как указано в презентации.

4. Блок скриншотов.

Мы показываем 3–5 ключевых экранов: календарь, карточка друга с анкетой, экран виш-листа, экран идеи подарка. Для каждого — короткая подпись, не больше двух строк.

Задача блока — снять «страх нового интерфейса» и показать, что приложение не сложное, а аккуратное и продуманное. Этот блок критичен для конверсии именно потому, что пользователи принимают решение глазами: картинка убеждает быстрее текста.

5. Блок «Модель использования и цена».

В MVP мы можем выбрать freemium с бесплатным базовым функционалом и платными расширениями (например, дополнительные напоминания, расширенные фильтры, шаблоны поздравлений).

На лендинге мы формулируем честно:

Сейчас Birthsync — в открытом тестировании.
Основной функционал доступен бесплатно. В будущем появятся премиум-функции, но базовые напоминания и анкеты останутся бесплатными.

Мы сознательно не ставим жёсткий ценник на лендинге в первый день, потому что на момент MVP мы ещё проверяем готовность платить. Но сразу проговариваем, что монетизация будет. Это лучше, чем сначала обещать «всё бесплатно навсегда», а затем неожиданно вводить платные тарифы.

6. Блок FAQ и доверие.

Кратко отвечаем на базовые вопросы: как хранятся данные, можно ли импортировать контакты, как удалить аккаунт.

Здесь же мы можем упомянуть, что стоим за продуктом как команда с конкретными именами (Мирослав, Роман, Полина, Виолетта) — это добавляет доверия и снижает страх перед передачей личной информации.

Для удобства преподавателя структуру лендинга можно свести в таблицу:

Блок	Содержание	Цель	Ответственный
Него	Название, слоган, СТА, фон-скриншоты	Сразу продать идею и дать путь	Полина (дизайн), Мирослав

Блок	Содержание	Цель	Ответственный
		действия	(формулировка)
Проблема	Краткое описание боли	Эмпатия, узнавание	Мирослав → Полина
Решение	3–4 ключевые преимущества	Показать, что боль закрывается	Мирослав + Роман (что реально уже есть)
Скриншоты	3–5 экранов с подписями	Снять страх интерфейса	Полина → Виолетта (проверка по факту билда)
Модель использования	Текущий этап, базовая бесплатность, намёк на премиум	Управление ожиданиями	Мирослав
FAQ и доверие	Приватность, команды, удаление данных	Снизить страх и возможные возражения	Роман (техчасть), Мирослав (формулировки)

Тогда давай скорректируем отчёт сразу под реальность: BirthSync = Telegram-бот сейчас + потенциальный Telegram Mini App, а не отдельные мобильные приложения. Ниже я перепишу и расширю именно то, что относится к Заданию №2 (упаковка) и №3 (инфраструктура), так, чтобы ты мог просто вставить это в отчёт вместо предыдущей версии.

(2.3.) Логика упаковки с учётом Telegram-формата

Мы исходим из того, что первая точка контакта пользователя с продуктом — это страница бота в Telegram и сообщения, через которые мы его приводим внутрь: посты, пересланные сообщения от друзей, упоминания в чатах. Отдельный внешний лендинг остаётся важным, но уже не как единственный вход, а как усилитель доверия и источник дополнительного трафика из поиска и соцсетей.

Поэтому стратегически мы ставим перед упаковкой три задачи:

1. Внутри Telegram максимально быстро и понятно объяснить, зачем запускать бота прямо сейчас, без лишних переходов и регистрации.
2. На внешнем лендинге дать более развёрнутое объяснение ценности BirthSync и ответить на вопросы про приватность, хранение данных и команду — то есть закрыть рациональные сомнения.
3. Сразу заложить основу под будущий Telegram Mini App: иконки, скриншоты, короткие тексты, которые можно будет использовать и как «витрину» Mini App, и как промо-материалы.

Эта логика важна, потому что в Telegram порог входа ниже, чем у классических приложений: пользователю не нужно ничего устанавливать, он просто нажимает «Start». Значит, packaging мы строим так, чтобы не мешать запуску лишними действиями, а не пытаться повторить модель App Store / Google Play один в один.

(2.4.) Структура «лендинга» внутри Telegram: профиль и первое сообщение

Мы рассматриваем профиль бота как мини-лендинг, который пользователь видит до нажатия Start. У нас есть несколько полей: имя, юзернейм, аватар, короткое описание и «about».

Мы заполняем их неформально, но предельно предметно.

Имя бота.

Мы используем имя вида: BirthSync — дни рождения и подарки. Такое имя одновременно сохраняет бренд и даёт понимание функции: пользователь сразу считывает, что речь о днях рождения и подарках, а не абстрактной продуктивности.

Аватар.

Иконка должна быстро читаться в маленьком круге. Мы не перегружаем её деталями: простой знак, ассоциирующийся с праздником и планированием (например, стилизованный календарь с ленточкой-подарком). Это важнее, чем красивая, но перегруженная иллюстрация, потому что в списке чатов и в результатах поиска Telegram пользователи принимают решения на уровне долей секунды.

Короткое описание.

Здесь мы даём одну–две строки, которые отвечают на вопрос «зачем мне это»:

«Храним дни рождения, предпочтения друзей и идеи подарков. Напомним заранее и поможем не дарить случайные вещи.»

Формулировка максимально утилитарна: мы не обещаем абстрактный «цифровой помощник», а сразу называем три сущности — даты, предпочтения, идеи подарков.

Поле “about” (если доступно).

Здесь мы немного расширяем формулировку и добавляем аспекты доверия:

«BirthSync — бот для планирования поздравлений. Никакого спама, только ваши контакты, ваши заметки и напоминания. Мы команда из четырёх человек, сами устали забывать важные дни и решили это исправить.»

Такой текст делает акцент на отсутствии спама и показывает, что за ботом стоят живые люди, а не анонимная «контора».

Первое приветственное сообщение после /start.

Это наш основной «Hero-экран» внутри Telegram. Мы строим его как аналог лендинга:

1. Одна фраза, описывающая ценность («Мы забираем с вас весь стресс с днями рождения.»).
2. Очень короткое объяснение, что бот делает прямо сейчас в MVP (календарь, анкеты, идеи подарков).
3. Одна понятная кнопка—СТА: «Создать первый день рождения» или «Импортировать контакты» (если используем какие-то полуавтоматические варианты).

Мы сознательно не даём пользователю пять кнопок и огромный текст в первом сообщении, чтобы не выключить его когнитивно. Наша цель — довести его до первого маленького «успеха» (создана одна запись), а уже внутри сценария пояснять остальное.

(2.5.) Внешний лендинг для Telegram-бота

Снаружи мы строим классический лендинг, но адаптируем его под Telegram-реальность.

Hero-блок выглядит не как «скачать приложение», а как «открыть бота»/«отправить себе ссылку в Telegram». Основной СТА — большая кнопка «Открыть бота в Telegram», вторичная — «Отправить ссылку себе» или «Оставить e-mail, чтобы не потерять».

С точки зрения содержания, мы оставляем те же смысловые блоки, что и в предыдущей версии отчёта (проблема, решение, для кого, FAQ), но в нескольких местах меняем акценты:

- Вместо обсуждения установки приложения объясняем, что не нужно ничего устанавливать, бот открывается в два клика. Это сильный аргумент для части аудитории, который снижает сопротивление.
- В блоке «Приватность» отдельно проговариваем, что BirthSync работает на базе Telegram, использует только те данные, которые пользователь добровольно вводит, и что у пользователя есть право удалить свои данные через команду/кнопку внутри бота. Это важно, потому что люди часто боятся «слива контактов».
- В FAQ добавляем вопросы про то, как бот взаимодействует с группами (можно ли использовать его в семейном чате, корпоративной группе и т.п.) и как дальше будет выглядеть Mini App-развитие (например, «Планируется ли визуальный интерфейс мини-приложения внутри Telegram?»).

Таким образом, лендинг становится не конкурентом, а усилителем Telegram-входа: часть пользователей зайдёт сразу через t.me-ссылку, часть — через сайт, но в обоих случаях «точкой истины» будет именно бот.

(2.6.) Упаковка под Telegram Mini App

Мы заранее проектируем визуал и тексты так, чтобы их можно было практически без переработки использовать в Mini App:

- Скриншоты, которые показываем на лендинге, делаем в стиле интерфейса будущего Mini App: карточка человека, виш-лист, календарь. Тогда, когда мы подключим Mini App, пользователи увидят знакомые экраны.
- Тексты внутри бота (подсказки, описания команд, подписи к кнопкам) пишем лаконичным продуктовым языком, потому что эти текстовые фрагменты фактически станут «микрокопирайтом» внутри Mini App.
- Аватар и название должны смотреться органично как в списке ботов, так и в списке Mini Apps, чтобы нам не пришлось переименовываться при масштабировании.

(2.7.) Текст анонса для запуска Telegram-бота

Мы перерабатываем анонс под конкретную механику Telegram:

Мы запустили BirthSync в Telegram — бота, который забирает с вас весь стресс с днями рождения.

Никаких установок, никаких сложных форм — просто открываете бота, добавляете близких и больше не вспоминаете о дате в последний момент.

Что умеет BirthSync сейчас:

- хранить дни рождения и напоминать заранее в личку;
- сохранять предпочтения друзей и «точно нельзя», чтобы не промахиваться с подарками;
- вести простые виш-листы и идеи, к которым можно вернуться в любой момент.

Это наш MVP в формате Telegram-бота. Мы планируем развивать его в сторону Mini App, с более богатым интерфейсом внутри Telegram.

Нам очень важна ваша обратная связь: что удобно, чего не хватает, что бесит. Пишите прямо в бота через раздел «Предложить идею».

👉 [Открыть BirthSync в Telegram](#)

Здесь мы подчёркиваем простоту входа («никаких установок»), явно называем текущий формат (бот), но отсылаем к будущему развитию (Mini App), чтобы сформировать ожидание эволюции, а не «застывшего» бота.

(3.) Инфраструктура релиза для Telegram-бота и Mini App

(3.1.) Общая архитектура с учётом Telegram

С переходом от нативного приложения к Telegram-боту ядро инфраструктуры с точки зрения бэкенда и данных остаётся похожим: нам по-прежнему нужен надёжный API, база данных, аналитика, мониторинг. Меняется клиентская часть и часть рисков.

Теперь у нас есть:

- Клиент уровня Telegram: бот, работающий через Bot API, который принимает апдейты и отправляет сообщения пользователям.
- Потенциальный клиент уровня Mini App: веб-интерфейс, который открывается внутри Telegram через web_app, считывая безопасный контекст от Telegram.
- Бэкенд, к которому обращается и бот, и Mini App (по защищённому HTTP API).
- Хранилище данных (PostgreSQL), которое знает всё о пользователях, контактах, анкетах и виш-листах.

Архитектуру мы по-прежнему строим как трёхслойную, но «клиентский слой» теперь — это Telegram вместо «голового» мобильного приложения.

(3.2.) Размещение бота и бэкенда

Мы размещаем основной бэкенд на облачном сервере с поддержкой HTTPS и публичным доменом, поскольку для работы через вебхуки Telegram требуется корректный TLS-сертификат.

Здесь у нас две возможные модели приёма апдейтов:

- Webhook. Telegram сам отправляет апдейты на наш HTTPS-эндпойнт. Это даёт минимальную задержку и экономит ресурсы, но требует стабильной доступности нашего сервера.
- Long polling. Наш бэкенд сам периодически опрашивает Telegram. Это менее требовательно к инфраструктуре, но хуже масштабируется при росте нагрузки.

Для MVP и особенно для будущего Mini App, где нам важна отзывчивость, мы склоняемся к модели webhook: она лучше подходит для случаев, когда пользователи активно взаимодействуют с ботом. Мы закладываем в архитектуру, что обработчики апдейтов не должны быть тяжёлыми: длительные операции (формирование дайджестов, массовые напоминания) выносятся в очередь задач и воркеры.

Таким образом, на уровне сервера у нас появляются:

- веб-сервер, принимающий webhook-запросы от Telegram и отдающий ответы (например, Nginx + приложение на Python/Node и т.п.);
- слой приложения, который реализует логику BirthSync (обработка команд, сценарии создания контакта, напоминаний);

- очередь задач и воркеры, которые отвечают за фоновые операции (массовая отправка напоминаний, вычисление статистики).

(3.3.) База данных и хранение данных Telegram-пользователей

База остаётся реляционной (PostgreSQL), с акцентом на целостность и безопасность персональных данных. Мы храним:

- telegram_id пользователя и метаданные (дата первого старта, язык интерфейса, настройки).
- Список контактов пользователя, привязанный к его аккаунту, с датами рождения и именами.
- Анкеты с предпочтениями и «нельзя».
- Виш-листы и идеи подарков.
- Логи событий для аналитики (создан контакт, заполнена анкета, добавлена идея, сработало напоминание).

Мы специально не привязываем структуру таблиц к особенностям Telegram (например, не полагаемся на никнеймы как на ключ), чтобы в будущем можно было подключить другие каналы или Mini App без полной переработки схемы.

Важный момент, специфичный для Telegram-бота: пользователи ожидают, что их данные можно удалить через сам бот. Поэтому в инфраструктуре мы проектируем жёсткую команду удаления (например, /delete_me или кнопку «Удалить аккаунт»), которая:

- помечает пользователя как удалённого;
- очищает/анонимизирует его контакты и заметки;
- при необходимости ставит короткий TTL для остаточных логов, чтобы они не висели «вечность».

С точки зрения резервного копирования мы продолжаем делать регулярные бэкапы PostgreSQL (не реже раза в сутки плюс point-in-time recovery), потому что ошибка в логике бота или в миграции может привести к повреждению данных, и нам потребуется вернуться назад.

(3.4.) Инфраструктура под Telegram Mini App

Mini App добавляет ещё один слой:

- Front-end Mini App — веб-приложение, раздаваемое нашим сервером как статика (HTML/CSS/JS), которое открывается внутри Telegram и получает контекст

пользователя через безопасный механизм `web_app_init_data`.

- API для Mini App — тот же бэкенд, который обслуживает бота, но с эндпойнтами, адаптированными под фронтенд Mini App (получение и обновление данных через AJAX/Fetch-запросы).

Для нас это удобно, потому что:

- логика и данные остаются едиными, бот и Mini App работают с одним и тем же ядром;
- мы можем постепенно мигрировать сценарии из «диалоговых» в «визуальные» без дублирования бизнес-логики.

С точки зрения инфраструктуры это означает, что мы должны заранее:

- позаботиться о CORS и авторизации для Mini App, чтобы запросы от web-интерфейса были безопасными и однозначно привязанными к конкретному Telegram-пользователю;
- убедиться, что производительность бэкенда достаточна для более «чаттерного» характера запросов от интерфейса (Mini App генерирует больше мелких запросов, чем обычный бот).

(3.5.) Аналитика и мониторинг в Telegram-контексте

Здесь мы дополняем ранее описанный контур аналитики спецификой Telegram:

- В продуктовую аналитику мы добавляем события Telegram-уровня: старт бота, возвращения пользователя, использование конкретных команд, открытие Mini App, завершение ключевых сценариев (например, «создан первый контакт»).
- Мы учитываем, что часть взаимодействий будет через inline-режим (если мы его введём, например, для быстрого выбора контакта прямо в чате). Это значит, что аналитика должна уметь различать сценарий «пользователь работает один на один с ботом» и сценарий «бот используется в чате/группе».

Технический мониторинг остаётся критически важным, но меняется набор рисков:

- Нужно следить за ошибками взаимодействия с Bot API (ограничения по скорости, ошибки сети, статусы 429). Мы закладываем логирование и алёрты по превышению `rate limit`, чтобы вовремя перестроить стратегию отправки напоминаний (например, разбивать массовые рассылки на пакеты).
- Для Mini App мы отслеживаем время ответа API и ошибки фронтенда (например, через фронтенд-логгер), потому что пользователь внутри Mini App ведёт себя как в обычном веб-приложении, и UX-ошибки сразу бьют по удержанию.

Таким образом, аналитика и мониторинг превращаются в наш основной инструмент согласования технических решений и продуктовой стратегии: мы видим, где пользователи реально «сыпятся» в сценариях Telegram-бота и Mini App, и под это подстраиваем релизы.

(3.6.) Выбор сервисов с учётом формата бота

Сервер и хостинг.

Мы выбираем облачный VPS или PaaS, который:

- даёт стабильный HTTPS (для вебхуков и Mini App);
- имеет предсказуемую ценовую модель (чтобы рост нагрузки не превратился в финансовый кризис);
- находится в юрисдикции, которая совместима с нашими юридическими планами по BirthSync.

База данных.

Управляемый PostgreSQL остаётся оптимальным выбором: мы не хотим своими силами решать задачи бэкапов, репликации и обновлений на стадии, когда команда ещё маленькая.

Очередь задач и кэш.

Для периодических напоминаний и массовых рассылок мы, по сути, обязаны использовать очередь (например, на базе Redis или RabbitMQ): без неё бот либо будет зависать, либо мы упрёмся в лимиты Telegram. Кэш (тот же Redis) также полезен для того, чтобы быстро хранить «горячие» данные (последние действия пользователя, состояние шагов сценария), не обращаясь каждый раз в БД.

Сервисы аналитики.

Часть аналитики мы можем вести через собственную базу (событийные таблицы), часть — через внешние решения, интегрированные с нашим бэкендом. Важнее не конкретная платформа, а дисциплина: каждое событие должно быть осмысленным, а не «случайным логом».

(3.7.) Кризисная устойчивость именно в Telegram-среде

Работа в Telegram накладывает дополнительные риски, которые мы учитываем в инфраструктуре релиза:

1. Риск блокировки или ограничений со стороны Telegram.

Мы выстраиваем логику напоминаний и рассылок так, чтобы не выглядеть как спам-

бот: никаких массовых сообщений без явного согласия пользователя, разумные лимиты уведомлений. Это снижает вероятность санкций.

2. Риск падения бэкенда при массовой активности.

Здесь критичны очереди и горизонтальное масштабирование: если мы понимаем, что нагрузка выросла (например, после удачного промо), мы можем добавить воркеры и увеличить мощность без полной остановки сервиса.

3. Риск негативного эффекта «кто-то переслал баг всем».

В Telegram любой баг или странное поведение может моментально разойтись по чатам. Поэтому мы сохраняем фокус на быстром rollback и честной коммуникации:

Роман отвечает за технический откат, Мирослав — за публичное сообщение, Полина — за формулировки, Виолетта — за фиксацию и анализ инцидента.

За счёт этого инфраструктура релиза BirthSync в формате Telegram-бота и Mini App остаётся не просто «набором серверов», а продуманной системой, где каждый элемент — от очереди задач до текстов в профиле бота — связан с конкретным риском и конкретной задачей продукта.